Relation Extraction from Natural Language

Anonymous ACL submission

Abstract

This project focuses on developing a deep learning model for relation extraction within the context of movie knowledge graphs. In this project use two datasets, `hw1_test-1.csv` and `hw1_train-1.csv`, The task involves pairing sentences(UTTERANCES) with their corresponding relationships(CORE RELATIONS). The model, built with a Multilayer Perceptron (MLP) architecture, is trained on `hw1_test-1.csv`, which includes sentences (UTTERANCES) and their associated relations (CORE RELATIONS). After training, the model predicts the CORE RELATIONS for each sentence (UTTERANCES) in `hw1_train-1.csv`. The resulting predictions are compiled into `submission.csv`, which serves as the final output of the assignment.

1 Introduction

This project aims to train a Deep learning model that matches sentences with corresponding knowledge graph relations from movies. This assessment is a practical exercise in the implementation of the NLP model. In this project, two datasets are utilized: `hw1_test-1.csv` and `hw1_train-1.csv`.

The `hw1_test-1.csv` dataset, which contains two columns—UTTERANCES (the sentences) and CORE RELATIONS (the corresponding relations)—is used for training the model. The model is trained to generate relationships based on the sentence and its associated relation provided in `hw1_train-1.csv`.

A Multilayer Perceptron (MLP) architecture is employed for this model. After training, the model predicts the CORE RELATIONS for each UTTERANCES in `hw1_train-1.csv`, and the results are saved in `submission.csv`, which contains the predictions based on `hw1_train-1.csv`.

2 Methodology

In this project involves several key Methodology steps, from importing necessary libraries to data pre-processing and device configuration for model training, MLP model, Hyperparameter Tuning.

2.1 Import Libraries

To begin, I imported various essential libraries for data manipulation, model building, and training. This includes:

- **Pandas**: Particularly for reading and managing CSV files.
- **NumPy**: Employed for numerical computations, especially for handling tensor and vector operations.
- **Torch**: PyTorch was used to define and train deep learning models.
- **Torch.nn** and **torch.optim**: Imported for defining neural network layers and optimizers, respectively.
- **sklearn.feature_extraction.text** and **sklearn.preprocessing**: Used for feature extraction and data preprocessing, particularly for text vectorization and handling multi-label data.
- **sklearn.model_selection**: Leveraged to split the dataset for model training and evaluation purposes.
- **torch.utils.data**: Utilized for creating PyTorch datasets and dataloaders, which facilitate batch training and streamline data management.

2.2 Device Configuration

To optimize computational resources, I configured the device to use a GPU when available.This step ensures that the model leverages the most efficient processing power available, reducing training time and enhancing performance.

2.3 Data Loading and Preprocessing

Reading and preprocessing a training dataset is critical in deep learning projects. I utilized the `pandas` library to load and manage the dataset (`hw1_train-1.csv`), which contains pairs of sentences (**UTTERANCES**) and their corresponding relations (**CORE RELATIONS**).

By leveraging the (`pd.read_csv()`) function, the dataset was loaded into a `pandas` DataFrame, allowing for efficient data manipulation and preprocessing. This step is foundational,preparing the data for subsequent transformation and training.

Data preprocessing involves identifying and handling missing values, as well as transforming multi-label relationships. Initially, I used the (`df.fillna("")`) function to replace any missing values in the **CORE RELATIONS** column with empty strings, ensuring data integrity. Subsequently, I transformed **CORE RELATIONS** into a list format to facilitate multi-label encoding.

2.4 Define X_{train} Feature Extraction with Bag-of-Words

To define X_{train} for the NLP model, this project employs the Bag-of-Words (BoW) approach. The Bag-of-Words model is a common technique for representing textual data in natural language processing (NLP) tasks. Since NLP algorithms cannot directly process raw text data, converting the text into a structured vector format is necessary. In this project I used the `CountVectorizer` function transforms the input text into a numerical vector based on the frequency of each word that appears within the dataset. Specifically, the `CountVectorizer` assigns a count to each word, capturing the term frequency across the corpus, which is beneficial for subsequent analysis. For this project, the maximum number of features I want is capped at 1000, indicating that only the 1000 most frequent words will be extracted as features.

Once the conversion is complete, the output serves as X_{train}, the input feature matrix for the model. Upon examination, X_{train} has a shape of (2312, 1000), which indicates that the dataset contains 2312 samples, each represented by a vector of 1000 features.

2.5 Multi-Label Encoding with MultiLabelBinarizer

To prepare Y_{train} for the NLP model, it is essential to understand the multi-label classifications present in the `CORE RELATIONS` column. A `MultiLabelBinarizer` was utilized to encode the categorical data into a binary format suitable for multi-label classification tasks.

The resulting Y_{train} matrix has a shape of (2312, 19), indicating that the dataset comprises 2312 samples annotated with 19 distinct labels. These labels encompass a diverse range of metadata attributes relevant to movies, including categories such as `actor.gender`, `movie.country`, `movie.directed_by`, and `movie.genre`. This dimensionality reflects the complexity and breadth of relational information within the dataset.

The `MultiLabelBinarizer` identified the following label classes:

$$\begin{bmatrix} \text{'actor.gender'} \\ \text{'gr.amount'} \\ \text{'movie.country'} \\ \text{'movie.directed_by'} \\ \text{'movie.estimated_budget'} \\ \text{'movie.genre'} \\ \text{'movie.gross_revenue'} \\ \text{'movie.initial_release_date'} \\ \text{'movie.language'} \\ \text{'movie.locations'} \\ \text{'movie.music'} \\ \text{'movie.produced_by'} \\ \text{'movie.production_companies'} \\ \text{'movie.rating'} \\ \text{'movie.starring.actor'} \\ \text{'movie.starring.character'} \\ \text{'movie.subjects'} \\ \text{'none'} \\ \text{'person.date_of_birth'} \end{bmatrix}$$

These labels provide a comprehensive framework for encoding relational attributes associated with movies and actors, facilitating a structured approach to multi-label classification within the NLP model. This encoding standardizes the relational data and supports the model's ability to process and classify complex, multi-faceted information relevant to cinematic entities and their properties.

2.6 Dataset Splitting

In this project, we used the `train_test_split` function to partition the dataset into training and validation sets. `test_size=0.2` allocates 20% of the data to the validation set and the remaining 80% for training.

This method allows for an initial assessment of the model during the training process, effec-

tively mitigating the risk of overfitting. The parameter `random_state=42` ensures that the split is reproducible, providing consistent results across different experimental runs. The resulting shapes, $X_{\text{train}} = (1849, 1000)$ and $X_{\text{val}} = (463, 1000)$, confirm that the dataset was split correctly.

2.7 Data Preparation and Loading

Deep learning models cannot directly process NumPy arrays or other data types. Therefore, it is essential to convert the data into PyTorch tensors. The initial step involves using the `torch.tensor()` function to create a PyTorch tensor from a NumPy array, with the data type specified as `torch.float32`, a commonly used floating-point format in deep learning. After converting to a tensor, the data is transferred to the designated device via `.to(device)`, typically a GPU. A GPU can significantly reduce training time due to its enhanced computational efficiency, especially with large datasets and complex models.

The `TensorDataset` class is then used to package the training and validation data into PyTorch-compatible datasets. `TensorDataset` pairs each feature tensor with its corresponding label tensor, forming a structured dataset where each sample consists of a feature-label pair. This format is essential for passing data into the `DataLoader`, which handles batch loading.

Finally, the `DataLoader` is employed to create data loaders, which facilitate efficient data handling with several key advantages: (1) **Batching**: Data is easily divided into batches of a specified size, which optimizes memory usage and computational efficiency. (2) **Shuffling**: Random shuffling of data helps prevent the model from memorizing the order of the data, thereby reducing the risk of overfitting. (3) **Automatic data loading**: `DataLoader` can automatically load data onto the GPU or CPU, streamlining the training process.

The `train_loader` and `val_loader` represent the training and validation datasets, respectively, with each training batch containing 32 samples. Notably, the `shuffle=False` parameter in `val_loader` ensures that the order of the validation data remains consistent, enabling accurate model evaluation.

3 Multilayer Perceptron Architecture

A multilayer perceptron (MLP) is a type of neural network that can function as either a feedforward or backpropagation network. It consists of an input layer, one or more hidden layers, and an output layer, enabling it to perform tasks such as classification and regression. In this model, I employed a feedforward propagation approach.

During the feedforward propagation process, input data flows from the input layer through one or more hidden layers and then to the output layer. Each neuron receives weighted inputs from all neurons in the preceding layer, calculates a weighted sum, and applies an activation function to produce its output.

Input Layer The input layer receives features from the dataset, with each neuron corresponding to a feature. For example, in `X_train`, each feature or word is represented as a neuron in the input layer.

Hidden Layers The MLP includes one or more hidden layers between the input and output layers. These layers are responsible for further processing the output of the input layer. Each neuron in these layers interacts through weighted connections and nonlinear activation functions, enabling the model to learn deeper features from the data. The depth (number of hidden layers) and width (number of neurons per layer) can be adjusted according to the task's complexity. In this example, the first hidden layer consists of 128 neurons, and the second hidden layer has 64 neurons.

Output Layer The output layer is the final layer in the MLP and is responsible for generating the model's predictions. The structure of this layer is determined by the specific task requirements. For example, a single neuron with a sigmoid activation function can be used for binary classification, while multiple neurons (often using softmax activation) are suitable for multi-class classification. For regression tasks, the output layer typically contains a single neuron that predicts a continuous value.

3.1 Key Parameters of the MLP Classifier

Hidden Layer Sizes (`hidden_layer_sizes`) This parameter defines the structure of the hidden layers, accepting a tuple where each element represents the number of neurons in a hidden layer. In this model, `hidden_layer_sizes` is set to (128, 64), meaning the first hidden layer has 128 neurons, and the second hidden layer has 64 neurons.

Activation Function (`activation`) The activation function introduces nonlinearity to each neu-

ron in the hidden layers, allowing the model to learn complex input-output mappings. Common activation functions include ReLU, tanh, and sigmoid. In this model, ReLU (Rectified Linear Unit) is used for its computational efficiency and ability to mitigate the vanishing gradient problem.

Learning Rate (`learning_rate`) The learning rate controls the magnitude of weight updates at each training iteration. Choosing the right learning rate is crucial for training stability and convergence. The rate can be "constant," "invscaling," or "adaptive," which affects how it changes over time. A high learning rate may cause the training to diverge, while a low rate can lead to slow convergence. Thus, the learning rate is considered a hyperparameter.

Additional Parameters The input dimension (`input_dim`) is set to the number of features, matching the number of columns in `X_train`. The output dimension (`output_dim`) is based on the number of classes for multi-label classification, which is 19 in this project. The model uses `BCEWithLogitsLoss` as the loss function, suitable for multi-label classification due to its combination of sigmoid activation and binary cross-entropy loss. The Adam optimizer is employed for weight updates with an initial learning rate of 0.001.

Feedforward Propagation The `MLPRelationModel` is initialized with three fully connected layers, `self.fc1`, `self.fc2`, and `self.fc3`, which sequentially process data from `input_dim` through the hidden layers to the output layer. ReLU activation and a dropout rate of 0.5 are used to prevent overfitting and enhance generalization.

4 Experiments And Hyperparameter Tuning Approaches

In machine learning model training, the need for hyperparameters varies depending on the dataset and task. Hyperparameter tuning is a highly technical process, as the only way to determine optimal hyperparameters is through multiple experiments, systematically testing different hyperparameter combinations to train the model. To achieve a high-performance model, this study employed various approaches for hyperparameter tuning and used statistical analysis (such as loss functions) to track and evaluate experimental results to identify the best hyperparameter settings.

In this research, I applied three different methods for hyperparameter tuning, including manual evaluation, grid search, and the automated tuning tool, **Optuna**:

- **Manual Evaluation**: Hyperparameters were adjusted based on the results of each experiment, incrementally refining them to improve model performance. I change the Learning rate from 0.0001 to 0.001, batch from 16 to 32 size, epoch around 50 to 500, Hidden Layer Sizes 128 to 512.
- **Grid Search**: This approach involved specifying possible values for hyperparameters and the performance metrics. The algorithm tried all possible combinations to identify the optimal hyperparameter set.
- **Optuna Automated Tuning**: Leveraging Optuna's dynamic tuning capability, this method iteratively optimized hyperparameters based on model performance. It efficiently explored a larger hyperparameter space and automatically eliminated unpromising combinations.

These strategies worked together to investigate and refine the model's hyperparameter settings, increasing the model's flexibility and prediction accuracy in datasets.

4.1 Grid Search for Hyperparameter Tuning

In this work, grid search is used as an exhaustive search approach, creating all conceivable hyperparameter combinations to evaluate model performance under various configurations. For this experiment, I used the `ParameterGrid` method to do grid search, with the Table 1 parameter range:

Hyperparameter	Values
Batch Size	16, 32, 64
Learning Rate	0.0001, 0.001, 0.005
Hidden Layer	(128,), (128, 64), (256, 128)
Epochs	100, 200, 300

Table 1: Parameter ranges used in Grid Search for hyperparameter tuning.

Within this range, all parameter combinations were thoroughly examined to determine the best configuration for model performance. Using this exhaustive search strategy, the best configuration achieved was Table 2, and Figure 1 shows the Best Model over 100 Epochs plot.

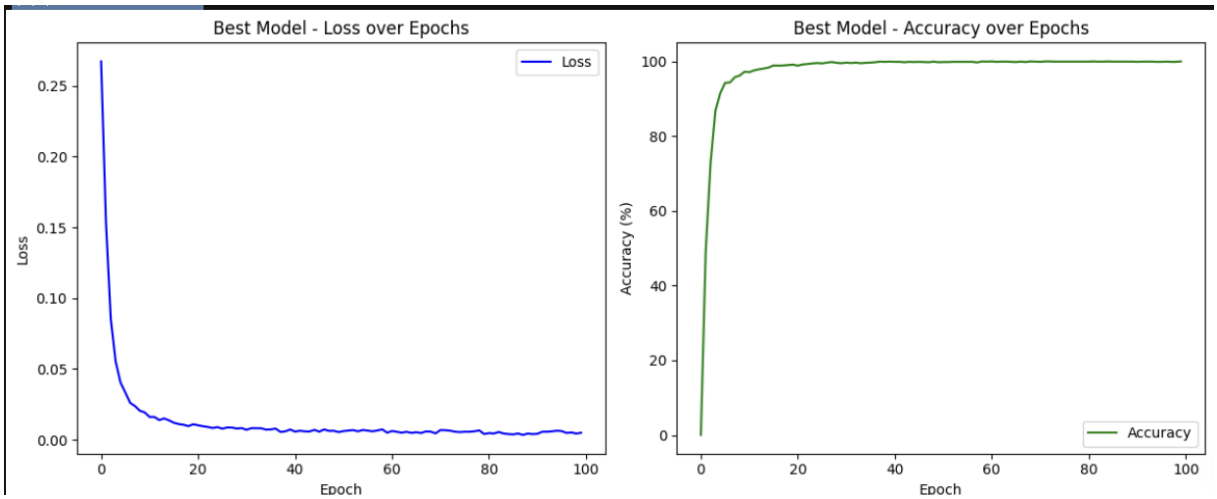

Figure 1: Performance of the Best Model over 100 Epochs. The left plot shows the loss decreasing over epochs, while the right plot illustrates the accuracy reaching near 100% and stabilizing.

Parameter	**Value**
Batch Size	16
Epochs	100
Hidden Layer Sizes	(256, 128)
Learning Rate	0.005
Accuracy	**99.95%**

Table 2: Best Grid Search Hyperparameters and Resulting Accuracy

4.2 Hyperparameter Optimization with Optuna

In this study, Optuna was also used for hyperparameter Optimization. Optuna is an open-source hyperparameter optimization framework designed to find the best hyperparameter combination within a predefined search space. The second part of the experiment, I used Optuna to perform search, with the following parameter ranges like Table 3 download below:

Hyperparameter	**Range/Values**
Batch Size	{16, 32, 64}
Learning Rate	10^{-5} to 10^{-2}
Hidden Layer Sizes	{(128,), (128, 64), (256, 128)}
Epochs	{100, 200, 300}

Table 3: Hyperparameter Ranges Search for Optuna

Within this parameter space, Optuna identified the optimal configuration as follows like the Table 4 :

Parameter	**Value**
Batch Size	32
Learning Rate	0.002741
Hidden Layer Sizes	(256, 128)
Epochs	100
Best Validation Accuracy	88.77%
Accuracy	**99.95%**

Table 4: Optimal Hyperparameters and Corresponding Accuracy

4.3 Visualization of the Optuna plot: Optimization History Plot and Hyperparameter Importances plot

The results of Optuna's evaluation include two main components: the **Optimization History Plot** and the **Hyperparameter Importances plot**.

4.4 Visualization of the Optuna plot: Optimization History Plot and Hyperparameter Importances plot

The results of Optuna's evaluation include two main components: the **Optimization History Plot** and the **Hyperparameter Importances plot**.

4.4.1 Optimization History Plot

The optimization progress over trials is displayed in this plot (Figure 2). After about ten trials, the objective value—which stands for model accuracy—stabilizes.

- **X-Axis (Trial)**: Represents different trial attempts, with each trial corresponding to a distinct set of hyperparameters.

- **Y-Axis (Objective Value)**: The objective value reflects model accuracy; a higher objective value indicates better model performance.

- **Blue Dots (Objective Value)**:The outcome of a particular combination of hyperparameters is represented by each blue dot.

- **Red Line (Best Value)**: The best objective value as of right now is shown by the red line. A good combination was found in this study as early as the tenth trial.

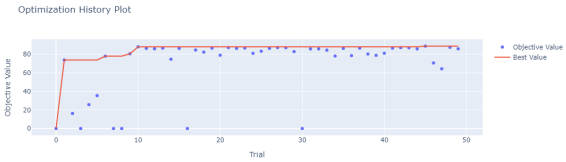

Figure 2: Optimization History Plot

4.4.2 Hyperparameter Importances

Optuna have the **(plot_param_importances)** function to reveal which hyperparameters have the most effects on performance. The hyperparameters examined in this study (*learning_rate*, *hidden_layer_sizes*, *batch_size*, and *epochs*) are indicated by the **Y-axis**. With 0.90 of the overall importance score, **learning_rate** had the biggest impact among them. This finding suggests that, in comparison to other parameters, adjusting the learning rate improves model performance more effectively. The Figure 3. illustrates the significance of each hyperparameter. Learning rate is the most important statistic because of its high percentage of importance.

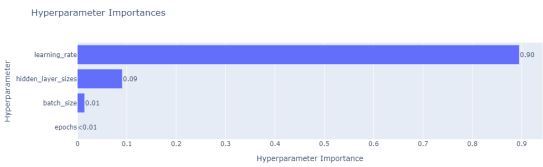

Figure 3: Hyperparameter Significance

5 Kaggle Results Comparison

In this experiment, I submitted three sets of best hyperparameter configurations to the Kaggle platform, each derived through different optimization methods:

- Using manually estimated hyperparameters, an accuracy of 0.79612 was achieved.

- The ParameterGrid exhaustive search method resulted in an accuracy of 0.78593.

- The Optuna automated hyperparameter tuning tool achieved the highest accuracy of 0.80326.

The findings clearly show that the combination of hyperparameters offered by Optuna produced the best accuracy of 0.80326, surpassing the accuracy of the other two approaches. Additionally, Optuna needed less time to determine the optimal set of hyperparameters. As a result, Optuna is a highly recommended solution for automated hyperparameter tuning since it can provide superior accuracy and function more efficiently.

Appendix

This appendix includes references to resources and libraries that were used or referenced throughout this project.

- **MultiLabelBinarizer Documentation**: Please refer to the official documentation for further information on managing multi-label classification: https://scikit-learn.org/dev/modules/generated/sklearn.preprocessing.MultiLabelBinarizer.html.

- **Tutorial on Natural Language Processing (NLP)**: For a thorough overview of NLP principles and methods, see here: https://www.geeksforgeeks.org/natural-language-processing-nlp-tutorial/.

- **Optuna**: Optuna, an open-source hyperparameter optimization framework, see the official site: https://optuna.org/.

- **ParameterGrid Documentation**: Understand the use of ParameterGrid for exhaustive hyperparameter searches, consult the official documentation here: https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.ParameterGrid.html.